From 3/13/2026-3/21/2026 I was in areas without access to internet and on vacation and thus was unable to work on the project. This was spring break. This has given me less time than normal to work and thus I may not be able to put in 10 hours. I will attempt to remedy this over the coming weeks.

Weekly Goals

These are somewhat anomalous. Given the break that just occurred, my current task has been to 'understand the model and iterate.' My main defined task is to understand the model. This is written before the model has been understood. I have a couple ideas for after which I will include here.

- Understand the AI model
    - Go through the files of the model we are using and understand interactions and functions
    - Experiment with model outputs to see if I can 'reverse engineer' the weight that it placed on specific variables
- Further iteration of the model
    - **Will be determined after the model is understood**
- Further data manipulation
    - **More goals may be determined after the model is understood**
    - Working on reducing data size while accurately representing ratios of crashes to non-crashes
    - Concatenate multiple cities to see how the AI model may be able handle scalability

Understanding the Model

This will be done with the assistance of general research and AI.

For the AI, I am using the one that was shown during the client meeting at the end of week 4. This seems to be the one the group is moving forward with. I did generate one at the end of last week and if I have time I may compare the two as, if my memory serves, the one generated by cursor on my end was much more complicated.

For understanding the model, this will be broken up into !) understanding the code and how it works 2) training the model and looking at its outputs.

Understanding the model

The following folders are in the ml directory which is where the model is trained/created.

- preprocessing.py
- model.py
- training.py

The additional directory of artifacts is held which I believe is just meant to hold the model. An evaluate.py is also located in order to give the status of the model after it has been generated. For now, my focus will just be on the three main files.

The command to train the model is:

```
```bash
.\venv313\Scripts\python -m ml.training --csv-path csv/TrainingData.csv --output-path
ml/artifacts/latest-model.pt --row-limit 250000 --epochs 8 --batch-size 512
```
```

Which means that it is trained on a virtual environment on the TrainingData.csv using the training.py inside the ml folder. This means that training is the starting point.

Using the variables provided, I do have some thoughts immediately. The standard seems to be that row-limits takes 250,000 rows of the data, introduces it in batches of 512, then does it 8 times over for the gradient (epochs).

While 8 epochs seems low, my biggest issue is the row-limit. While the data needs to be cut down at some point, the current training data has three million lines of data that is used to build history. This means the AI is trained on only 8 percent of that data. I am worried that with such a low percentage, something like city/road history will not be able to be reflected in any meaningful way. This might also mess up weather as snow (a rare weather event) may not even be reflected inside the AI.

I will reserve final judgement until I go through the model completely (as well as train it as it may be a reaction to long training times). I also already need to cut down on training data size but given that the goal of the project is something that can scale, this worries me.

training.py

From my understanding, and just common logic, this is what will generate the AI. This is what takes in the arguments from the above command such as row-limit, epochs, and batch size and then begins the actual training. This is likely going to have preprocessing.py run early and then have model.py run late. I will list those as they come up which may make this section of the report read less than linear.

The main function in training.py first makes an argos variable through _build_parser()

- _build_parser

This is just taking the information from the argument line provided and then adds them to a parser so that other functions can use the information provided.

- run_training_pipeline

Main run this next. This is basically the main function for most of the file. Any function from here on out (minus the status text) is called from this function.

The first thing this does is call load_training_dataframe which is a function imported from preprocessing.py . This is documented under preprocessing.py. The result is the data that will be used to train the model and stored in a variable called dataframe

After the dataframe is built, a variable called shape is built from the build_feature table which is imported from preprocessing.py. Just as with load_training_dataframe, this will be documented under preprocessing.py in thai document.

Next is split_feature_table also from the preprocessing file. The notes for this will be under the preprocessing section.

Next in the pipeline is to_tensor_dataset which is once again, imported from preprocessing.py and will be under the pipeline_to_tensor bullet inside of preprocessing.py

At this point, the preprocessing file has been gone through and there should be no further references to it. Now training can begin. After the preprocessing of the data is done, a model is created (model.py) and stored inside of a variable called model. Next is training

- train_binary_classification

This is where I am going to need a lot of assistance from others or AI as this is not where my experience lies.

At first glance, this is an expanded version of the basic training loop from the video I was looking over in the week 4 report. The data is shuffled and then fed into the batches. Most of it seems abstracted as it was in the video from last week. The gradient is calculated and general adjustments are made.

The batches do seem to be a ‘secondary epoch’ in a way where each batch also causes recalculation and reshifting of the model’s weights.

In terms of data generation, which is where most of my work on this project has been and where my mind continues to be, it does seem to have some sort of balancing for the ratio of positive to negative samples which is good. I am curious as to how that is going to work when I attempt to query the AI myself. I am not worried, but I do wonder how that will affect things like snow and ice which are naturally going to have a much higher rate of crashes than when the world is sunny and clear. There is a world where the balancing removes the difference, but we will see I may be catastrophizing.

- tune_threshold

This is a threshold that adjusts for false positives/negatives. If, based on the training data it is unlikely to predict a crash or too likely to predict a crash based off the TrainingData we can adjust it slightly to ensure that is either more or less willing to predict a crash. This feels somewhat linear though.

I was under the impression that we were to get a probability instead of a physical yes or no answer as to if there would be a crash or not. A yes/no does little use to us if the goal is to color code roads by risk. Looking at the README though, I do think there are other issues.

<u>Aside</u>

From this point on, it looks like the rest of the pipeline is just saving the output of the training to the artifacts directory to be queried later. There is no need to go over that as it shouldn't actually affect how the model works, or if it does it is outside the realm that I wish to explore at present.

preprocessing.py

This seems to format the data. This defines a couple of categories of data

```
NUMERIC_FEATURE_COLUMNS = ["crashmonth", "crashyear", "crashtime_minutes", "hour_bucket"]
CATEGORICAL_FEATURE_COLUMNS = [
    "city",
    "county",
    "dayofweek",
    "roadclass",
    "ruralurbantype",
    "surfacecondition",
    "weathercondition",
]
```

This is important for feature engineering later in the file.

- load_training_dataframe

This seems to be what picks out the data inside of the TrainingData.csv. This confirms that the TrainingData file exists, then runs

```
  loaded = pd.read_csv(csv_file, nrows=row_limit)
```

This, to my understanding, only reads the first *row_limit* lines. This is a large issue. The limited amount of lines provided by the default command resulted in around 8 percent of the data actually being trained on. If I am understanding the code right, this means it's only the first 8 percent of the data. So maybe only January and then some of February. Given the goal was to build the history of roads this means that we are very far off from the goal.

Aside from the issue, once it checks to make sure that no columns are missing inside of the data (a check that can be made redundant in the data generation step) before sending it back to training.py

- build_feature_table

This seems to decide general categories to train the data under. Based on the dataframe created in the last function.

This seems to create crashtimeminute and hour_bucket as generalizations of the data inside the training data. This step can be done in the training data to save time/remove complexity in this step if needed.

Afterwards it then flattens all the numeric and categorical features into readable types. This is going to create a giant map of all the possible counties, days of week, and so on. So it will likely result in something like:

Monday: 0.0
Tuesday: 0.0
Wednesday: 1.0
Thursday: 0.0
…

Which will massively inflate the data. I do not yet know if there is a better way to store it, but that can be looked into to save space and ensure that things run faster.

The other issue is that I don't think this maps StreetName at all. Which means that the history we are building for streets currently does not exist.

I will want to double check this during model training as it will likely clear everything up, but that is the guess based on research.

- split_feature_table

This splits the data into what is going to go into training and what is going to have the model test against. An 80|20 split which means of the 250,000 rows of training data, 50,000 is going into the test. This does have what goes into each category randomized which is really good, but means that it still is over the first two months at best. Once this is done, it sends this back to training.py along with the marker of which is which.

X should be data to train on while Y is designed to provide the model with a basis to correct itself. Leaning the model weights in the correct direction.

- to_tensor_dataset

Where the feature table made the values numbers, this just turns the training data into tensors or tables of data that PyTorch is going to be able to train off of. Not much is going on here.

model.py

This is, as I understand, a very simple ML model that has three layers. My general knowledge about how layers work is still lacking. After going over the rest of how this model works, I will do my best to look into more about layers and see if the model might benefit from more layers or different types of layers (I think I read somewhere that it was possible). However, this is going to be a general what if type of question along with the idea of running more epochs and likely depends heavily upon how long the model takes to train as it stands right now.

Running the model

At this point I have identified a couple of problems that I want to address when it comes to making the data/model slightly better. Mostly in regards to how the TrainingData is generated and run. The next thing to observe any other issues in the model would be to train and then run the model.

Getting the stack correct to train the model took some time, but the actual model training time took around 2-3 minutes. More epochs/layers could certainly be used to increase accuracy.

Querying the model is simple which is nice, but there is a rather big problem with the model as of present. At the very least the README and swagger testing environment ask for the model to be provided with:

```
{
  "latitude": 32.7767,
  "longitude": -96.7970,
  "road_name": "Main St",
  "segment": "Dallas city center",
  "road_class": "CITY STREET",
  "weather_condition": "1 - CLEAR"
}
```

Where only road_class and weather_condition are aspects that the ai is trained on. Latitude, longitude, and segment are not in the TrainingData.csv yet though there are plans to bring them there eventually.

Based on just playing with the model for a while, I don't think anything is changing the score as it is now. I acknowledge that I may be using swagger wrong, but this score:

```
 "risk_score": 0.474595,
```

Is all I have gotten despite modifying every single value inside of the query. This

Possible Improvements

General improvements I believe could be made. I assume most of these issues are real, but I do acknowledge that some of these issues may not be real or may be unnecessary. Given that this is not the model I generated/made, I am not going to make any changes on anything to do with the model until after the meeting on 3/26/2026 at least.

The data generation is what I have been working on and can be done either before or after the meeting. However, given the general progress I am making, this may need to wait until week 6.

- Perhaps increasing the rows the model is trained on as a default
- Shuffle rows so that the model is not trained only on the first x (250,000 right now) rows and is instead trained with data located all over the TrainingData file.
    - May need to be done inside of the data generation step
- Find a way to reduce the amount of space training will take up
    - May not be possible
- Incorporate streets inside the training data
    - May need to switch to street numbers instead of names
    - Possible issue of lookup tables for the main application may be needed. Worth looking into
- Look into better model training methods
    - Incredibly vague but worth looking into if there is an excess of time in the coming weeks.
- Improve documentation of model
    - Write in model training updates. Estimates of when things will be done maybe

9/26/2026 Meeting

This is the weekly team meeting where we prepare for the client meeting on Friday that is done every week.

Before Meeting

Most of the above was written before the meeting. Running the model will continue after this meeting and will be listed below section. As of 20 minutes before the meeting begins I have topics to bring up, mainly how we want to proceed on the model. The Possible Improvements is the list of issues I intend to bring up. Based on what everyone has done this week, the direction we want to go with the project and tasks for next week will result in a general plan of action before the meeting on 9/27/2026.

After Meeting

General progress on model development seems to be research, similar to what I have been doing. Apparently a second model is in development that might be able to perform similar or better to the current model. Model iteration seems to be the main prerogative of at least three of us. Thus I want to spend at least three more hours working on this project (totalling to around 8).

Given that features are still needed, I want the bulk of my time to be around data generation. More specifically I want to explore removing redundant data, simplifying some of the data that exists, and using average traffic numbers to reduce data size while keeping ratios of data preserved. I want to wait until the second AI model is finished, but one of the issues that I mentioned was the mapping of strings, ie. creating flags for each individual string like

Monday: 1.0
Tuesday: 0.0
…

While the answer is very likely no, I would like to see if there is some version of a model that can use road numbers instead of names. I don't think this will solve the issue without creating many more issues, but it is worth going over in my opinion.

The second thing I can do is continue to go over the existing model. I would like to make some adjustments to prove that I have worked on something tangible. However, I think adding good documentation and then verifying model performance would be a good use of time. That way, once the second model has been developed, the performance of both can be compared and a single model can be chosen between the two.

Second Round of Data Generation

My primary goal is scalability. I have mentioned in several of my reports that the size of the data is unsustainable, thus I want to see if I cut down on the size of the model before I make changes to the data that exists. As a reminder, C.R.I.S. is the source of the crash reports:

https://cris.dot.state.tx.us/public/Query/app/query-builder/disclaimer

For the weather report I am using NOAA

https://www.ncei.noaa.gov/access/search/data-search/global-hourly

Given that NOAA has only updated their 2025 reports up to 8/25/2025, I am going to move to 2023 or 2024 data for the TestingData.csv. A large file of 2020-2025 data has been ordered and delivered from C.R.I.S. but now needs to be split up and delivered to members of the group for use

In order to do this, I will need to convert anywhere from 6-18 csv files into one. Given that 2024 for Dallas County is just over the threshold this is unavoidable if we seek to ensure that there is scalability based on C.R.I.S. 's data. I will look for a way to automate this in the coming weeks so that an easy tool exists, but for now I will do general notation off of 2025 data.

Currently testing data is formatted like this

```
Crash ID,City,County,Crash Date,Crash Month,Crash Time,Crash Year,Day of Week,Hour of
Day,Road Class,Rural Urban Type,Street Name,Surface Condition,Weather Condition,Crash
```

Crash ID is redundant and needs to be removed, the way that Crash Hour is formatted needs to be changed, Rural Urban Type may be a needless column given how many rows do have it defined. Additionally, most of the models and mapping tools being implemented by the project currently query for and provide a road segment which needs to be added.

I also want to cut down on the amount of data that is gathered so I will be attempting to look at a couple of new columns. These include: average daily traffic amount, average daily traffic year.

Average daily traffic year seems to just be the year, thus for now the only extra field that we will keep will be average daily traffic. This issue with this field is that often there is no data, so some roads may not be able to be measured.

The new pulled data is this:

```
"Crash ID","Average Daily Traffic Amount","City","County","Crash Date","Crash
Month","Crash Time","Crash Year","Day of Week","Hour of Day","Road Class","Rural Urban
Type","Section","Street Name","Surface Condition"
```

While the AI is being trained I will add more features as opposed to taking them away, removal will be easier later. CrashID can be removed but is a required column of data generation.

Crash ID can be removed from the data at the start of the TrainingData process. Average Daily Traffic will need to be calculated later to see how (and if we can) use it to calculate better row amounts for the negative samples. The section will likely need to be treated as a secondary road which will completely redo how many roads the data will be looking at once.

The next step is to through csv overview again and look at: How many roads have segments, how many roads have average daily traffic amounts and if that number is the same, if the number is not the same what is the average of the daily traffic amounts, what is the new average amount of roads with section added in, as well as perhaps looking at exactly how many roads have a Rural Urban Type. This will likely be all that is possible for this week but should be able to help shape how the next week is going to be done and what tasks I am to take during that time.

- Section Overview

One thing I was worried about was there would be very little roads that had sections or too many that had sections. Basically the section tag would do nothing for the data or would double the size of the data. The below is the comparison of checking just roads and the roads and sections

```
Total unique roads: 4275
Total unique road-section pairs: 4322
```

This is still a small amount of extra roads made through sections, but are 47 sections mostly for the highway which is good I suppose and may become more pronounced outside of more urban areas. I am worried about how this might affect the average calculations of crashes. Also, it might be good to merge roads and sections so that the AI model will not try to learn things like ‘section 3 is the most dangerous section for all roads.

- Average Daily Traffic Amount

The result is for average daily traffic amount that there are a lot of rows with average daily traffic data:

```
Total rows                :  48809
Rows WITH ADT data        :  20798  (42.6%)
Rows MISSING ADT data     :  28011  (57.4%)
```

There are not a lot of roads with average daily traffic data. The function average_daily_traffic_overview used road plus section information which resulted in in the 4322 roads from above

```
Roads with NO ADT data across all their rows (4243 roads):
```

This means there are only 79 roads with average daily traffic amounts which is not enough to do anything with. Initial output suggested that this was mostly for roads with 1 - 5 crashes which would be generalized under ‘Non Urgent Roads’ anyway (the concept being another large issue with this idea anyway), there are some daily large roads that do not have traffic information

```
ABRAMS                                    (1 crash row)
ABRAMS RD                                 (140 crash rows)
ABSHIRE LN                                (1 crash row)
```

While I still do like the plan of using traffic numbers to calculate, it is clear from this information that there is just not enough data here to go off of. I will think about this more, but for now I don't know if this information can be used in any meaningful way.

Next Week Tasks

Given that there is a second model being developed, I want to hold off on iterating on the models for now, so my primary task will be to continue working on the next round of data generation.

- Clean up current data generation
  - Remove crash id from training data
  - Make hour just a standard hour (1 to represent 1:00 - 1:59)
  - Look into how necessary crash time (ex: 1:32) is in comparison to just the hour
- Continue looking into average crash number calculations
  - Maybe find a new source for the data
  - Perhaps a county traffic ratio or even just a road type average traffic ratio
- Create some sort of file that will convert information from the large C.R.I.S. csv folders into one usable csv file. (This will take a long time)

Once a model has been decided on, more tasks can be come up to iterate on the model, but even just addressing the possible improvements in my Possible Improvements sections just above the 9/26/2026 meeting would be a good first step.